

List , Tuples, Dictionary and Functions

List

- ❖ Collection of Multiple Data.
- ❖ Holds Multiple Data types.
- ❖ Lists are Mutable.
- ❖ Store multiple data at same time.
- ❖ Creating a list is as simple as putting different comma-separated values between square brackets.

For example:

```
a = [1, 12.5, 'a', "python"]
```

```
list1 = ['physics', 'chemistry', 1997, 2000];
```

Accessing Values in Lists

To access values in lists, use the square brackets for slicing along with the index or indices to obtain value available at that index.

For example:

```
a = [1, 12.5, 'a', "python", "Programming"]  
print(a[1])  # Output is 12.5  
print(a[3])  # Output is python  
print(a[1:4]) # Output is [12.5, 'a', 'python']
```

Accessing Values in Lists

Example:

```
L = [9,18,'Hi',12,"Hello",15.55,'Programming',100,125.5]
```

```
print(L[5])
```

```
print(L[1:5])
```

```
print(L[2:8])
```

```
print(L[2:9:3])
```

```
print(L[-1])
```

```
print(L[-5])
```

Accessing Values in Lists

Example:

```
L = [9,18,'Hi',12,"Hello",15.55,'Programming',100,125.5]
```

```
print(L[0:9:3]) # Here '3' is step parameter
```

Now you also write the above code as

```
print(L[::3])
```

Here both **print(L[0:9:3])** and **print(L[::3])** gives an output as `[9, 12, 'Programming']`

Updating Lists

Lists in Python are **mutable**. All that means is that after defining a list, it is possible to update the individual items in a list.

You can update single or multiple elements of lists by giving the slice on the left-hand side of the assignment operator.

You can also add to elements in a list with the **append()** method.

Updating Lists

Example:

```
z = [30, 17, 40, 2]
```

```
# Update the item at index 1 with the string  
"python"
```

```
z[1] = "Python"
```

```
print(z) # Output->[30, 'Python', 40, 2]
```

```
z.append(100) #Use of append method to add 100
```

```
print(z) # Output->[30, 'Python', 40, 2, 100]
```

Deleting List Elements

To remove a list element, you can use either the *del statement* or *remove()* method.

For Example:

```
b = ['Python', 100, 'Programming', 2, 'is']  
del b[1] # deleting element of 1st index  
print(b) # ['Python', 'Programming', 2, 'is']  
b.remove(2) # Removing the element '2'  
print(b) # ['Python', 'Programming', 'is']
```

Basic List Operations

Lists respond to the `+` and `*` operators much like strings; they mean **concatenation** and **repetition** here too, except that the result is a new list, not a string.

Let's see some of the basic list operations in Python.

Built-in List Functions and Methods

Python includes following list **methods**:

list.append(obj): Appends object obj to list

list.count(obj): Returns count of how many times obj occurs in list

list.index(obj): Returns the lowest index in list that obj appears

list.insert(index, obj): Inserts object obj into list at offset index

list.pop(obj=list[-1]) : Removes and returns last object or obj from list

Built-in List Functions and Methods

Python includes following list **methods**:

list.remove(obj): Removes object obj from list

list.reverse(): Reverses objects of list in place

list.sort([func]): Sorts objects of list, use compare func if given

list.extend(seq): Appends the contents of seq to list

Tuple

A tuple is a sequence of **immutable** Python objects. Tuples are sequences, just like lists. The differences between tuples and lists are, the tuples cannot be changed unlike lists and tuples use parentheses, whereas lists use square brackets. A single tuple may contain **Data Types** like Integers, Strings, as well as Objects. Creating a tuple is as simple as putting different comma-separated values. Optionally you can put these comma-separated values between parentheses also.

Accessing Values in Tuples

To access values in tuple, use the square brackets for slicing along with the index or indices to obtain value available at that index.

For example:

```
t = (10, 122.5, "A", "Data-Science", "Python")  
print(t[1])  # Output is 122.5  
print(t[3])  # Output is 'Data-Science'  
print(t[1:4]) # Output is (122.5, 'A', 'Data-Science')
```

```
a = (1, 12.5, 'a', "python", "Programming")
print(a[1])      # Output is 12.5
print(a[3])      # Output is python
print(a[1:4])    # Output is [12.5, 'a', 'python']
```

```
T = (9, 18, 'Hi', 12, "Hello", 15.55, 'Programming', 100, 125.5)
print(T[5])
print(T[1:5])
print(T[2:8])
print(T[2:9:3])
print(T[-1])
print(T[-5])
print(T[0:9:3])
print(T[::3])
print(T[2::2])  # ['Hi', 'Hello', 'Programming', 125.5]
print(T[2::3])  # ['Hi', 15.55, 125.5]
print(T[3::3])  # [12, 'Programming']
print(T[5:-2])  # [15.55, 'Programming']
print(T[-2:-5:-1])  # [100, 'Programming', 15.55]
print(T[-2:-7:-2])  # [100, 15.55, 12]
```

Can you update Tuples element?

Tuples are **immutable** which means you cannot **update** or **change** the values of tuple elements. You are able to take portions of existing tuples to create new tuples as the following example demonstrates:

```
tup1 = (10, 4.6);
```

```
tup2 = ('Hi', 'Hello');
```

```
# Following action is not valid for tuples
```

```
tup1[0] = 100;
```

```
# So let's create a new tuple as follows
```

```
tup3 = tup1 + tup2;
```

```
print(tup3);
```

Deleting Tuple Elements

Removing individual tuple elements is not possible. But you can delete the entire tuple. To explicitly remove an entire tuple, just use the **del** statement.

For example:

```
tup = ('Math', 'English', 85, 65);  
print (tup);  
del tup;  
print (tup);
```

Basic Tuples Operations

Tuples respond to the **+** and ***** operators much like strings; they mean **concatenation** and **repetition** here too, except that the result is a new tuple, not a string.

Let's see some of the basic tuple operations in Python.

```
#Basic tuple operation
b = (10.5, 15.5)
c = (1, 2, 3, 4, 5, 6)
print(len(c))
print(b+c)
print(c*3)
print(3 in c)
print(40 not in c)
```

Built-in Tuple Functions

Python includes the following tuple **functions**:

cmp(tuple1, tuple2) : Compares elements of both tuples.

Please note **cmp()** does not support in python 3.

len(tuple): Gives the total length of the tuple.

max(tuple):Returns item from the **tuple** with max value.

min(tuple): Returns item from the **tuple** with min value.

Let's see the example of tuple functions.

```
#Example of tuple function
#print(cmp(b,c))
print(len(c))
print(max(c))
print(min(c))
li = (10,20,15,666)
print(len(li))
print(max(li))
print(min(li))
```

Tuple Methods

Python has **two built-in methods** that you can use on tuples.

count(): Returns the number of times a specified value occurs in a tuple

index(): Searches the tuple for a specified value and returns the position of where it was found.

Let's see the example of tuple methods.

#Example of tuple method

```
d = (20,15,10,'Hello','Python',10,10)
```

```
print(d.count(10))
```

```
print(d.index(10))
```

Difference Between List and Tuple:

SR.NO.	LIST	TUPLE
1	Lists are mutable	Tuple are immutable
2	Implication of iterations is Time-consuming	Implication of iterations is comparatively Faster
3	The list is better for performing operations, such as insertion and deletion.	Tuple data type is appropriate for accessing the elements
4	Lists consume more memory	Tuple consume less memory as compared to the list
5	Lists have several built-in methods	Tuple does not have many built-in methods.

Dictionary

A dictionary is a collection of data items which are unordered, changeable and indexed. In Python dictionaries are written with curly brackets, and they have keys and values.

Each key is separated from its value by a colon (:), the items are separated by commas, and the whole thing is enclosed in curly braces. An empty dictionary without any items is written with just two curly braces, like this: {}.

Dictionary

Keys are unique within a dictionary while values may not be. **The values of a dictionary can be of any type, but the keys must be of an immutable data type such as strings, numbers, or tuples.**

More than one entry per key not allowed. Which means no duplicate key is allowed. When duplicate keys encountered during assignment, the last assignment wins .

For example:

```
D = {'Name': 'James', 'Age': 35, 'Degree': 'Master'};
```

Accessing Values in Dictionary

To access dictionary elements, you can use the familiar square brackets along with the key to obtain its value. Following is a simple example:

```
D = {'Name': 'James', 'Age': 35, 'Degree': 'Master'};
print (D['Name'])
print (D['Degree'])
print (D['Age'])
```

If we attempt to access a data item with a key, which is not part of the dictionary, we get an error

```
D = {'Name': 'James', 'Age': 35, 'Degree': 'Master'};
print(D)
print(D['Name'])
print(D['Degree'])
print(D['Age'])
```

```
D['Age'] = 28; # update existing entry
D['School'] = "DPS"; # Adding new entry
print(D)
```

```
del D['Name'] # remove entry with key 'Name'
D.clear() # remove all entries in dict
del D # delete entire dictionary
```

#Dictionary Methods

```
d3 = {'Roll_no': 45, 'class': 6, 'Percentage': 85.5}
d4 = d3.copy()
print(d4)
```

$$H_1 = \{ \text{H}_1, \text{H}_2, \text{H}_3, \text{H}_4, \text{H}_5, \text{H}_6, \text{H}_7, \text{H}_8, \text{H}_9, \text{H}_{10}, \text{H}_{11}, \text{H}_{12}, \text{H}_{13}, \text{H}_{14}, \text{H}_{15}, \text{H}_{16}, \text{H}_{17}, \text{H}_{18}, \text{H}_{19}, \text{H}_{20}, \text{H}_{21}, \text{H}_{22}, \text{H}_{23}, \text{H}_{24}, \text{H}_{25}, \text{H}_{26}, \text{H}_{27}, \text{H}_{28}, \text{H}_{29}, \text{H}_{30}, \text{H}_{31}, \text{H}_{32}, \text{H}_{33}, \text{H}_{34}, \text{H}_{35}, \text{H}_{36}, \text{H}_{37}, \text{H}_{38}, \text{H}_{39}, \text{H}_{40}, \text{H}_{41}, \text{H}_{42}, \text{H}_{43}, \text{H}_{44}, \text{H}_{45}, \text{H}_{46}, \text{H}_{47}, \text{H}_{48}, \text{H}_{49}, \text{H}_{50}, \text{H}_{51}, \text{H}_{52}, \text{H}_{53}, \text{H}_{54}, \text{H}_{55}, \text{H}_{56}, \text{H}_{57}, \text{H}_{58}, \text{H}_{59}, \text{H}_{60}, \text{H}_{61}, \text{H}_{62}, \text{H}_{63}, \text{H}_{64}, \text{H}_{65}, \text{H}_{66}, \text{H}_{67}, \text{H}_{68}, \text{H}_{69}, \text{H}_{70}, \text{H}_{71}, \text{H}_{72}, \text{H}_{73}, \text{H}_{74}, \text{H}_{75}, \text{H}_{76}, \text{H}_{77}, \text{H}_{78}, \text{H}_{79}, \text{H}_{80}, \text{H}_{81}, \text{H}_{82}, \text{H}_{83}, \text{H}_{84}, \text{H}_{85}, \text{H}_{86}, \text{H}_{87}, \text{H}_{88}, \text{H}_{89}, \text{H}_{90}, \text{H}_{91}, \text{H}_{92}, \text{H}_{93}, \text{H}_{94}, \text{H}_{95}, \text{H}_{96}, \text{H}_{97}, \text{H}_{98}, \text{H}_{99}, \text{H}_{100} \}$$

Delete Dictionary Elements

You can either remove individual dictionary elements or clear the entire contents of a dictionary. You can also delete entire dictionary in a single operation.

```
D = {'Name': 'James', 'Age': 35, 'Degree': 'Master'};
```

```
del D['Name'] # remove entry with key 'Name'
```

```
D.clear() # remove all entries in dict
```

```
del D # delete entire dictionary
```

Built-in Dictionary Functions and Methods

Python includes the following dictionary functions:

len(dict) : Gives the total length of the dictionary. This would be equal to the number of items in the dictionary.

str(dict) : Produces a printable string representation of a dictionary.

type(variable) : Returns the type of the passed variable. If passed variable is dictionary, then it would return a dictionary type.

Built-in Dictionary Functions and Methods

Python includes following dictionary methods:

dict.clear() : Removes all elements of dictionary *dict*.

dict.copy() : Returns a shallow copy of dictionary *dict*

dict.fromkeys() : Create a new dictionary with keys from *seq* and values *set* to *value*.

dict.get(key, default=None) : For key *key*, returns value or default if key not in dictionary

Built-in Dictionary Functions and Methods

Python includes following dictionary methods:

dict.items() : Returns a list of *dict*'s (key, value) tuple pairs.

dict.keys() : Returns list of dictionary dict's keys

dict.setdefault(key, default=None) :

Similar to get(), but will set dict[key]=default if *key* is not already in dict

dict.update(dict2) : Adds dictionary *dict2*'s key-values pairs to *dict*

dict.values() : Returns list of dictionary *dict*'s values

```
#Dictionary Methods
d3 = {'Roll_no': 45, 'class': 6, 'Percentage': 85.5}
d4 = d3.copy()
print(d4)
#usage of fromkeys() method.
seq = ('name', 'age', 'sex')
dict = dict.fromkeys(seq)
print(dict)
dict = dict.fromkeys(seq, 10)
print(dict)

print(d3.get('class')) #Use of get method
print(d3['class'])
print(d3.items())

print(d3.keys())
print(d3.values())
print(d3.setdefault('class', None))
```

What is a Function?

A function is a set of statements sorted out together to play out a specific task. Python has a enormous number of in-built functions and the user can also make their own functions.

Functions are utilized to sensibly break our code into less complex parts which become simple to keep up and comprehend.

A developer develops a function to abstain from rehashing a similar assignment, or reduce complexity.

What is a Function?

- In Python, function is a **group of related statements** that perform a specific task.
- Functions help break our program into **smaller** and modular chunks.
- Furthermore, it avoids **repetition** and makes **code reusable**.

Defining a Function

Syntax of a Function:

```
def function_name(parameters):  
    statement(s)
```

Example of Function:

```
def pow (x, y) :  
    result = x**y  
    print(x,"raised to the power", y, "is", result)
```

Here, we created a function called **pow()**.

It takes two arguments, finds the first argument raised to the power of second argument and prints the result in appropriate format.

Calling a Function

Defining a function only gives it a name, specifies the parameters that are to be included in the function and structures the blocks of code.

Once a function is defined, you may call by its **function name** with the required number of arguments/parameters.

Following is the example to call **pow()** function:

```
def pow (x, y) :  
    result = x**y  
    print(x,"raised to the power", y, "is", result)
```

```
pow(5,3)
```

```
def pow(x, y):  
    result = x**y  
    print(x, "raised to the power", y, "is", result)
```

```
pow(5, 3)
```

```
def sum(n1, n2):  
    s = n1 + n2  
    return s
```

```
x = int(input("Enter the number: "))  
y = int(input("Enter the number: "))  
print(sum(x, y))
```

```
sum = lambda n1, n2 : n1 + n2  
x = int(input("Enter the number: "))  
y = int(input("Enter the number: "))  
print(sum(x, y))
```

Ways to write a function

With
argument with
return type

With
argument
without
return type

Without
argument with
return type

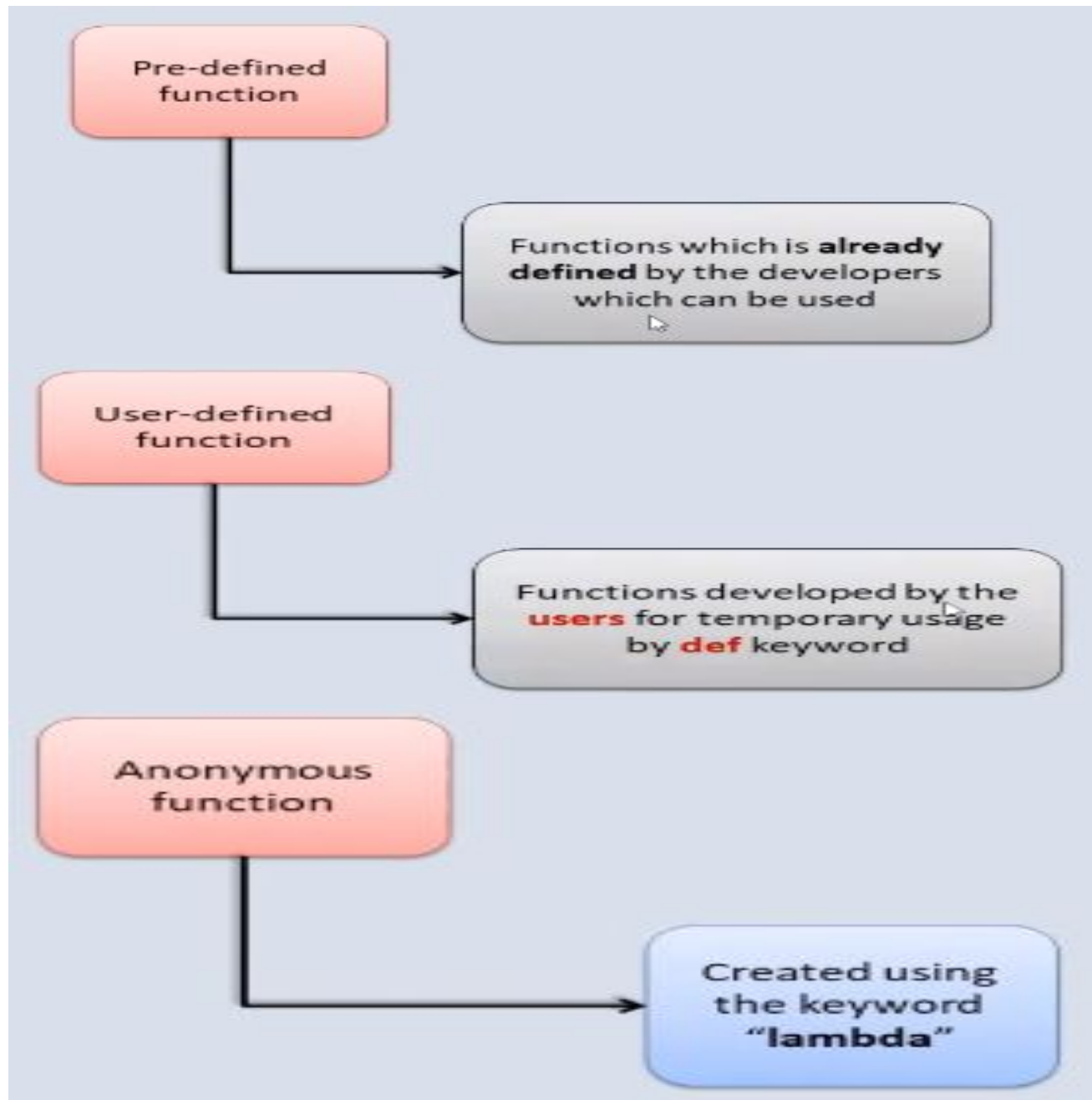
Without
argument
without return
type

Types of Functions

Pre-defined
function

User-defined
function

Anonymous
function



Anonymous Functions

These functions are called **anonymous** because they are not declared in the standard manner by using the *def* keyword. You can use the *lambda* keyword to create small anonymous functions.

- Lambda forms can take **any number of arguments** but **return just one value** in the form of an expression.
- They **cannot** contain **commands** or **multiple expressions**.
- An anonymous function **cannot be a direct call to print** because **lambda requires an expression**

Anonymous Functions

Program to add two numbers using function

```
def sum(n1,n2):  
    s=n1+n2  
    return s  
x=int(input("Enter the  
number"))  
y=int(input("Enter the  
number"))  
print(sum(x,y))
```

Program to add two numbers using lambda function :

```
sum = lambda n1,n2 :  
    n1+n2  
x=int(input("Enter the  
number"))  
y=int(input("Enter the  
number"))  
print(sum(x,y))
```

```
sum = lambda n1,n2 : n1+n2
x=int(input("Enter the first number: "))
y=int(input("Enter the second number: "))
print(sum(x,y))
print(sum(15,30))
```

```
mul = lambda n1,n2,n3 : n1*n2*n3
x=int(input("Enter the first number: "))
y=int(input("Enter the second number: "))
z=int(input("Enter the third number: "))

print(mul(x,y,z))
print(mul(10,12,20))
```

Recursive Functions

A function that calls itself is called a **recursive function** and this technique is known as **recursion**. This special programming technique can be used to solve problems by breaking them into smaller and simpler sub-problems.

Recursion is a common mathematical and programming concept. It means that a function calls itself. This has the benefit of meaning that you can loop through data to reach a result.

```
#Recursive function
def fact(n):
    if(n==1):
        return 1
    else:
        return n*fact(n-1)
print("Factorial of number is", end=" ")
print(fact(5))
```

What is a Module?

A module allows you to logically organize your Python code. Grouping related code into a module makes the code easier to understand and use. A module is a Python object with arbitrarily named attributes that you can bind and reference. Simply, a module is a file consisting of Python code. A module can define functions, classes and variables. A module can also include runnable code.

Creating a module

To create a module just save the code you want in a file with the file extension **.py**

For example:

A simple module, **Calculator.py**

```
def add(x, y):  
    return (x+y)
```

```
def subtract(x, y):  
    return (x-y)
```

```
def multiply(x, y):  
    return (x*y)
```

import Statement

You can use any Python source file as a module by executing an import statement in some other Python source file.

Now we can use the module we just created, by using the **import** statement:

When using a function from a module, use the syntax: ***module_name.function_name.***

```
import Calculator
```

```
print (Calculator.add(100, 12))
```

```
print (Calculator.multiply(100, 12))
```

The *from...import* Statement

Python's *from* statement lets you import specific attributes from a module.

For Example:

```
from math import sqrt, factorial
```

```
# if we simply do "import math", then
```

```
# math.sqrt(16) and math.factorial() are needed
```

```
print (sqrt(16))
```

```
print (factorial(6))
```

Re-naming a Module

You can create an alias when you import a module, by using the **as** keyword:

For Example:

```
import Calculator as clc  
print (clc.subtract(100, 12))  
print (clc.multiply(10, 12))
```

The *from...import ** Statement

It is also possible to import all names from a module into the current namespace by using the following import statement:

from modulename import *

This provides an easy way to import all the items from a module into the current namespace.

For Example:

from math import *

print(pow(5,3))

print(radians(30))

print(sqrt(36))

```
import Calculator as C
print (C.add(100, 12))
print (C.multiply(10, 12))
```

```
# Import built-in module math
print("Math module")
import math
print(dir(math))
```

```
#from--import *
from math import *
print(pow(5,3))
print(radians(30))
print(sqrt(36))
print(degrees(0.5239))
```

```
import math
print(math.pow(5,2))
print(math.factorial(5))
```

Locating modules

The module search path is stored in the system module `sys` as the **`sys.path`** variable. The `sys.path` variable contains the current directory, `PYTHONPATH`, and the installation-dependent default.

By the way, we can find the current working directory of python as follows:

```
import os  
print(os.getcwd())
```